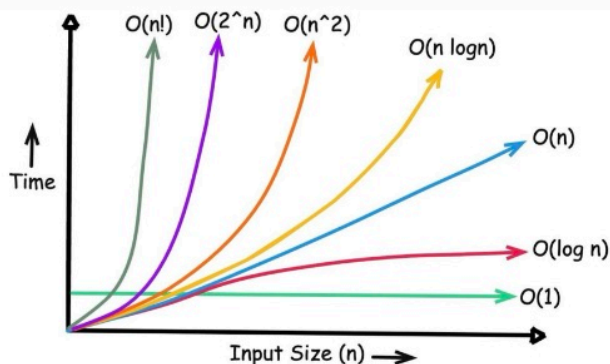




O	Ω	Θ
Big Oh	Omega	Theta
Upper Bound	Lower Bound	Upper & Lower Bound
Worst Case	Best Case	Average 'ish'

1. Circle/mark all the statements below that are true:

a. $\log(n) = O(n)$ $\log n < n$

b. $n = O(\log(n))$ $n < \log n$

c. $\log^2(n) = O(n)$ $\log n < n$

d. $n = O(\log^2(n))$ $n < \log n$

e. $\log(n) = \Omega(n)$ $\log n > n$

f. $n = \Omega(\log(n))$ $n > \log n$

g. $5n^3 + 7n + 13 = O(n^5)$ $5n^3 < n^5$

h. $5n^3 + 7n + 13 = \Omega(n^5)$ $n^3 > n^5$

i. $5n^3 + 7n + 13 = \Omega(n)$ $n^3 > n$

j. $\log(n!) = \Theta(n \log n)$ $\log(n!) = \Theta(n \log n)$ $\log(n!) \leq O(n \log n)$ $\geq \Omega(n \log n)$ $\left. \begin{array}{l} \text{Both upper \& lower bounds are the same} \\ \text{true} \end{array} \right\}$

k. $n^{1/2} = O(\log(n))$ $n^{1/2} < \log n$

l. $2^n = \Omega(n!)$ $2^n > n!$ $\rightarrow$ Burns

Answer the following:

m. big-Oh runtime for Merge Sort = $O(n \log n)$

n. big-Oh runtime for Selection Sort = $O(n^2)$

o. big-Oh runtime for Quick Sort = $O(n^2)$

p. lower bound on comparison-based sorting problem = $\Omega(n \log n)$

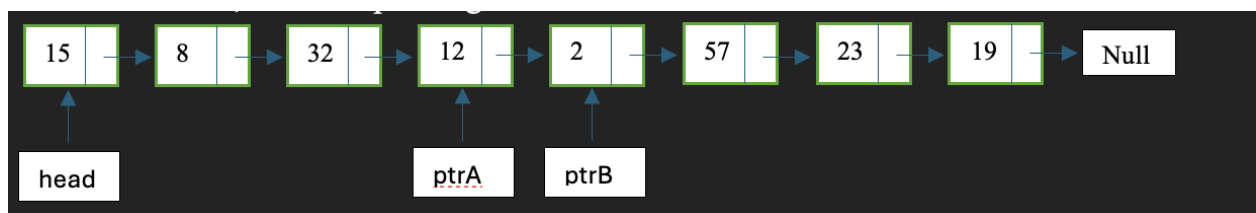
q. big-Oh runtime for binary search in a sorted array = $O(\log n)$

Linked List

For these questions, the data type 'node' refers to the type defined by the class

```
class node
{
public:
    int data;
    node * next;
};
```

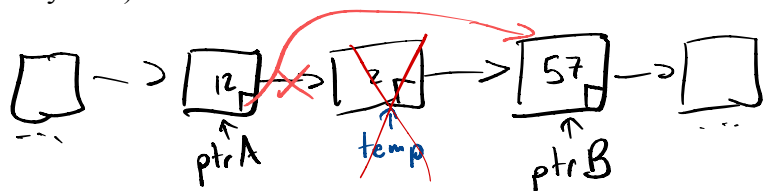
Consider the linked list below with node pointers ptrA and ptrB pointing to the specified nodes in the list, and head pointing to the first node in the list.



- a. Assume ptrB points to the node with 2 in the data field and ptrA points to the node previous to that node. Write the C++ statements that delete the node containing 2 and update ptrB so that it points to the node that followed 2 after deletion.

(Your solution must not create a memory leak)

```
ptrA -> next = ptrB -> next;
node* temp = ptrB;
ptrB = ptrB -> next;
delete temp;
```



- b. Assume ptrB points to the node with 2 in the data field and ptrA points to the node previous to that node.

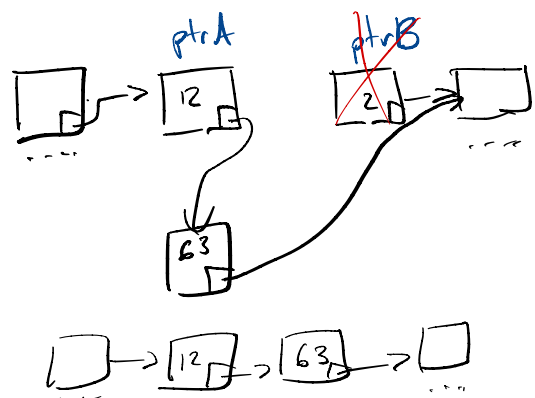
Write the C++ statements that:

Delete the node containing 2

Insert a new node containing 63 in the same position where 2 was removed

(Your solution must not create a memory leak.)

```
node* newNode = new node;
newNode -> data = 63;
newNode -> next = ptrB -> next;
ptrA -> next = newNode;
delete ptrB;
```



Sorting

For this question, suppose you have a `linkedList` class that includes methods 'split' and 'merge', already implemented (exactly as was done in the homework):

```
class linkedList{
```

```
Private:
```

```
    class node{
```

```
    Public:
```

```
        double data;
```

```
        node * next;
```

```
        node * prev;
```

```
    };
```

```
    node * head; //A pointer to the first node in the list
```

```
    node * tail; //A pointer to the last node in the list
```

```
Public:
```

```
    //Splits the contents of list A evenly into the two given (initially empty) lists.
```

```
    //If the number of items in the list is odd, puts the extra item in the first list.
```

```
    //Runs in O(n) time, where n is the total number of items in the list.
```

```
    void split(linkedList &A, linkedList &B);
```

```
    //Merge takes 2 sorted lists and merges them
```

```
    //into your (initially empty) list to create one sorted list.
```

```
    //Runs in O(n) time, where n is the total number of items in the two given lists.
```

```
    void merge(linkedList &A, linkedList &B);
```

```
};
```

a. Given this linked list class and the methods `split` and `merge`, add a method `sort()` to the `linkedList` class that sorts the list into ascending order using a divide-and-conquer strategy. (You may use the methods `split` and `merge` to help achieve this.)

(If the list contains fewer than 2 elements, your method should return immediately.)

b. Give a recurrence equation describing the runtime of your sorting algorithm.

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

c. Provide the Big-Oh runtime of your sorting algorithm.

$$O(n \log n)$$

a) void linkedList::sort()
{
 if (head == nullptr || head->next == nullptr)
 return;
 linkedList A, B;
 split(A, B);
 A.sort();
 B.sort();
 merge(A, B);
}

Binary Search Tree

For the following question, assume binary trees consist of nodes from the following class:

```
class node {  
    public:  
        double data;  
        node * left;  
        node * right;  
        node (double x) {data = x; left = nullptr; right = nullptr; }  
};
```

Write a C++ function 'node* buildTree(vector<double> &A, int start, int end)' that takes as input a sorted vector A, and creates a binary search tree using the elements in A from indices start to end, inclusive. The function should return a pointer to the root of the newly created tree, and the tree's height must be $O(\log n)$, where n is the number of items in the tree.

```
node* buildTree (vector<double> &A, int start, int end)
```

```
{
```

```
// Base Case: no elements
```

```
if ( start > end)
```

```
    return nullptr;
```

```
// Choose middle element
```

```
int mid = start + (end - start) / 2;
```

```
// Create root node
```

```
node* root = new node (A[mid]);
```

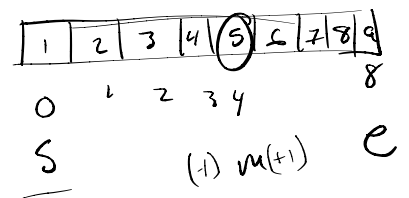
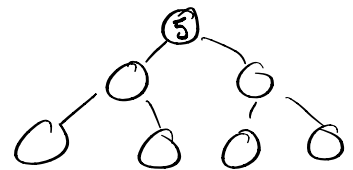
```
// Recursively build left & right subtrees
```

```
root->left = buildTree(A, start, mid - 1);
```

```
root->right = buildTree(A, mid + 1, end);
```

```
return root;
```

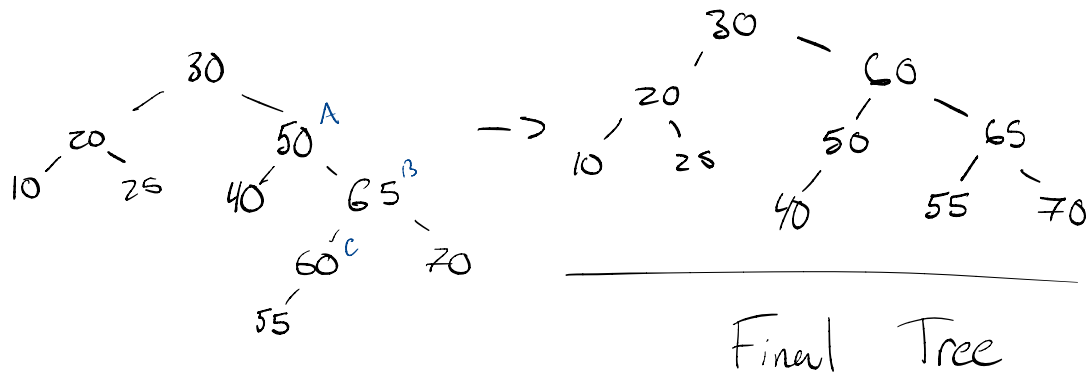
```
}
```



AVL Tree

a. Given the following sequence of values inserted in the order shown into an initially empty AVL Tree:

40, 20, 60, 10, 30, 25, 50, 70, 65, 55



b. Suppose the following AVL Tree node class is given:

```
class AVLNode {
```

```
Public:
```

```
    int data;
```

```
    AVLNode* left;
```

```
    AVLNode* right;
```

```
    int height;
```

```
};
```

Implement the following:

(You may assume helper functions such as getHeight() are allowed.)

1. Function to compute and return the balance factor of a node (check difference between left and right subtree): `int getBalance(AVLNode* node)`
2. Right rotation function and return root of subtree: `AVLNode* rightRotate(AVLNode* p)`
3. Left rotation function and return root of subtree: `AVLNode* leftRotate(AVLNode* p)`
4. BONUS: Insert() function (inserts a value into the AVL Tree and restores balance using rotations when necessary)

// Suppose the following AVL Tree node class is given:

```
class AVLNode {
public:
    int data;
    AVLNode* left;
    AVLNode* right;
    int height;
};
```

// An empty subtree has height 0.

```
int getHeight(AVLNode* node) {
    if (node == nullptr)
        return 0;
    return node->height;
}
```

// Recalculate height after changing one of a node's children.

```
void updateHeight(AVLNode* node) {
    int leftHeight = getHeight(node->left);
    int rightHeight = getHeight(node->right);

    if (leftHeight > rightHeight)
        node->height = leftHeight + 1;
    else
        node->height = rightHeight + 1;
}
```

// 1. Function to compute and return the balance factor of a node (check difference between left and right subtree):

```
// int getBalance(AVLNode* node)
int getBalance(AVLNode* node) {
    if (node == nullptr)
        return 0;

    return getHeight(node->left) - getHeight(node->right);
}
```

// 2. Right rotation function and return root of subtree:

```
// AVLNode* rightRotate(AVLNode* p)
AVLNode* rightRotate(AVLNode* p) {
    AVLNode* A = p; // The unbalanced node.
    AVLNode* B = A->left; // B moves up to become the new root.
    AVLNode* C = B->right; // C is moved between A and B.

    B->right = A;
    A->left = C;

    // A is now below B, so update A first.
    updateHeight(A);
    updateHeight(B);

    return B;
}
```

// 3. Left rotation function and return root of subtree:

```
// AVLNode* leftRotate(AVLNode* p)
AVLNode* leftRotate(AVLNode* p) {
    AVLNode* A = p; // The unbalanced node.
    AVLNode* B = A->right; // B moves up to become the new root.
    AVLNode* C = B->left; // C is moved between A and B.

    B->left = A;
    A->right = C;

    // A is now below B, so update A first.
    updateHeight(A);
    updateHeight(B);

    return B;
}
```

Helpers!

```
// 4. BONUS: Insert() function (inserts a value into the AVL Tree and restores
// balance using rotations when necessary)
```

```
AVLNode* insert(AVLNode* root, int value) {
    // First, do a normal binary-search-tree insertion.
    if (root == nullptr)
        return new AVLNode{value, nullptr, nullptr, 1};

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else // This version ignores duplicate values.
        return root;

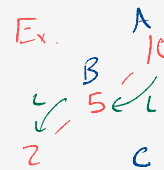
    // Update height
    updateHeight(root);
    int balance = getBalance(root);
```

Insert
Recursively

Then update node heights and perform necessary rotations as we work our way back up the recursive stack.

```
// Left-Left: left side too tall; value added left of left child.
// Fix: rotate right.
```

```
if (balance > 1 && value < root->left->data) {
    return rightRotate(root);
}
```

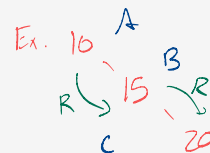


Right rotation



```
// Right-Right: right side too tall; value added right of right child.
// Fix: rotate left.
```

```
if (balance < -1 && value > root->right->data) {
    return leftRotate(root);
}
```

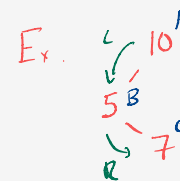


Left Rotation



```
// Left-Right: left side too tall; value added right of left child.
// Fix zig-zag: left rotate child, then right rotate root.
```

```
if (balance > 1 && value > root->left->data) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
```

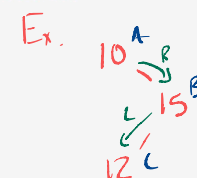


LR rotation

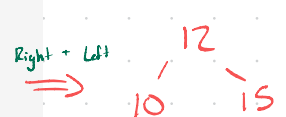


```
// Right-Left: right side too tall; value added left of right child.
// Fix zig-zag: right rotate child, then left rotate root.
```

```
if (balance < -1 && value < root->right->data) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
```



RL rotation



```
return root;
```

```
}
```